

Implementierung: Projektsoll-Cutting für cflux Zeiterfassung

Kontext

In cflux müssen Arbeitszeiten nach **Projektsoll-Vorgaben** "gecutet" werden, damit nur die tatsächlich dem Kunden verrechenbare Zeit abgerechnet wird. Ein Mitarbeiter kann außerhalb der Projektsoll-Zeit stempeln (z.B. zu früh kommen oder zu spät gehen), aber nur die Zeit innerhalb des Projektsoll-Fensters ist abrechenbar.

Geschäftslogik

Problem

Beispiel:

- Projektsoll: 06:00 - 17:00 (10h netto nach Pause)
- Mitarbeiter stempelt: 05:25 - 17:35 (12h 10min brutto)
- Differenz: 1h 10min (35min zu früh + 35min zu spät)

Anforderung:

- Für **Lohnabrechnung**: Volle gestempelte Zeit bezahlen (12h 10min)
- Für **Kundenrechnung**: Nur Projektsoll verrechnen (10h)
- Die 1h 10min Differenz ist "Mitarbeiters Bier" und wird nicht dem Kunden berechnet

Zwei-Konten-System (Schweizer Arbeitsrecht)

Arbeitszeitkonto (AZK):

- Enthält alle Überstunden über Normalarbeitszeit (z.B. 40h/Woche)
- Beispiel: 50h gearbeitet bei 40h Normal → 10h im AZK

Überzeitkonto (ÜZK):

- **Ist ein Indikator**, kein separates Konto!
- Markiert Stunden über gesetzlicher Grenze (45h/Woche in Schweiz)
- Diese Stunden bekommen 25% Zuschlag
- Beispiel: Bei 50h Arbeit → 5h (50-45) werden als zuschlagsberechtigt markiert

Abbau-Logik (LIFO):

- Bei Ersatzruhetag oder Stundenabbau werden **zuerst die zuschlagsberechtigten Stunden** (ÜZK-Indikator) abgebaut
- Restliche Stunden werden vom AZK abgezogen
- Beispiel: 8h Ersatzruhetag bei AZK=10h, ÜZK=5h
 - Erst ÜZK: 5h eliminiert
 - Dann AZK: 3h abgezogen

- Ergebnis: AZK=2h, ÜZK=0h

Zuschläge

In der Schweiz zuschlagsberechtigt:

- Nachtarbeit (22:00 - 06:00): +25%
- Sonntagsarbeit: +50%
- Feiertagsarbeit: +100%
- **KEIN** Samstagszuschlag in der Schweiz

In Deutschland zusätzlich:

- Samstagszuschlag: +50%

"Nicht zuschlagsberechtigt"-Flag:

- Mitarbeiter kann Stunden als "nicht zuschlagsberechtigt" markieren
- Use Case: Private Nacharbeit am Samstag/Sonntag → kein Zuschlag
- Grund muss angegeben werden

Wichtig: Zuschläge werden nur auf die **abrechenbare Zeit** berechnet, nicht auf die gesamte gestempelte Zeit!

Anforderungen an die Implementierung

1. Prisma Schema erweitern

Zeitmodell

Erweitere das **Zeitmodell** um:

- **tagesSollStunden** (Decimal): Standard-Tages-Sollarbeitszeit (z.B. 8.4h bei 42h/Woche)
- **projektsollAktiv** (Boolean): Ob Projektsoll-Cutting aktiv ist
- **projektsollFlexibel** (Boolean): Ob Projektsoll pro Projekt unterschiedlich sein kann
- Zuschlagsdefinitionen:
 - **nachtBeginn** (String): z.B. "22:00"
 - **nachtEnde** (String): z.B. "06:00"
 - **nachtZuschlag** (Decimal): z.B. 0.25 (= 25%)
 - **sonntagZuschlag** (Decimal): z.B. 0.50
 - **feiertagZuschlag** (Decimal): z.B. 1.00
 - **samstagZuschlag** (Decimal): z.B. 0.00 (CH) oder 0.50 (DE)

Projekt

Erweitere das **Projekt** um Projektsoll-Vorgaben:

- **sollBeginn** (String): Geplanter Arbeitsbeginn (z.B. "06:00")
- **sollEnde** (String): Geplantes Arbeitsende (z.B. "17:00")
- **sollPauseDauer** (Int): Geplante Pause in Minuten (z.B. 60)

- **sollArbeitszeit** (Decimal): Netto-Arbeitszeit in Stunden (z.B. 10.0)
- **cuttingAktiv** (Boolean): Ob Cutting für dieses Projekt aktiv ist
- **cuttingTolerance** (Int): Toleranz in Minuten (z.B. 0 = keine Toleranz)

Zeiterfassung

Erweitere die **Zeiterfassung** um:

Projektsoll-Zeiten (vom System berechnet):

- **sollBeginn** (DateTime): Berechneter Soll-Beginn
- **sollEnde** (DateTime): Berechnetes Soll-Ende
- **sollPause** (Int): Soll-Pause in Minuten

Cutting-Ergebnisse:

- **abrechenbareStunden** (Decimal): Stunden die dem Kunden berechnet werden
- **nichtAbrechenbar** (Decimal): Stunden außerhalb Projektsoll
- **vorSoll** (Int): Minuten zu früh gekommen
- **nachSoll** (Int): Minuten zu spät geblieben

Zuschlagsstunden:

- **nachtStunden** (Decimal): Stunden in Nachtzeit
- **sonntagStunden** (Decimal): Stunden am Sonntag
- **feiertagStunden** (Decimal): Stunden an Feiertagen
- **samstagStunden** (Decimal): Stunden am Samstag (nur Deutschland)

Zuschlagsberechtigungs-Flags:

- **zuschlagNacht** (Boolean, default true): Nachtarbeit zuschlagsberechtigt?
- **zuschlagSonntag** (Boolean, default true): Sonntagsarbeit zuschlagsberechtigt?
- **zuschlagFeiertag** (Boolean, default true): Feiertagsarbeit zuschlagsberechtigt?
- **zuschlagSamstag** (Boolean, default true): Samstagarbeit zuschlagsberechtigt? (nur DE)
- **zuschlagGrund** (String): Grund wenn nicht zuschlagsberechtigt (z.B. "Private Nacharbeit")

Indizes hinzufügen:

- `@@index([mitarbeiterId, datum])`
- `@@index([projektId, datum])`

2. Cutting Service implementieren

Erstelle einen **CuttingService** der folgende Funktionen bereitstellt:

Hauptfunktion: **cutZeiterfassung(zeiterfassungId: string)**

Input: ID einer Zeiterfassung **Output:** **CuttingResult** Objekt mit:

- **gesamtGestempelt**: Gesamte gestempelte Zeit in Minuten
- **abrechenbar**: Abrechenbare Zeit in Minuten

- **nichtAbrechenbar**: Nicht abrechenbare Zeit in Minuten
- **vorSoll**: Minuten vor Projektsoll-Beginn
- **nachSoll**: Minuten nach Projektsoll-Ende
- **nachtStunden**: Stunden in Nachtzeit
- **sonntagStunden**: Stunden am Sonntag
- **feiertagStunden**: Stunden an Feiertagen

Ablauf:

1. Zeiterfassung mit Projekt und Mitarbeiter (inkl. Zeitmodell) laden
2. Prüfen ob Stempelzeit beendet ist (sonst Fehler)
3. Projektsoll ermitteln (siehe unten)
4. Abrechenbare Zeit berechnen (siehe unten)
5. Zuschläge berechnen (siehe unten)
6. Ergebnisse in Zeiterfassung-Datensatz speichern
7. Result zurückgeben

Hilfsfunktion: **ermittleProjektsoll(zeiterfassung)****Logik:**

1. Wenn Projekt **sollBeginn** und **sollEnde** definiert hat:
 - Verwende diese Zeiten
 - Kombiniere mit Datum der Zeiterfassung
 - Erstelle DateTime-Objekte
2. Fallback: Wenn Projekt kein Soll hat:
 - Verwende Zeitmodell des Mitarbeiters (**tagesSollStunden**)
 - Standardannahme: 08:00 Beginn, berechne Ende basierend auf Sollstunden
 - Standardpause: 60 Minuten

Output: Objekt mit:

- **beginn**: DateTime des Soll-Beginns
- **ende**: DateTime des Soll-Endes
- **pause**: Pause in Minuten

Hilfsfunktion: **berechneAbrechenbar(zeiterfassung, projektsoll)****Logik:**

1. Gestempelte Gesamtzeit berechnen:
 - **stempelEnde - stempelBeginn - Pausen**
2. Abrechenbare Zeit berechnen:
 - Finde Überschneidung zwischen gestempelter Zeit und Projektsoll
 - **arbeitsBeginn = MAX(stempelBeginn, sollBeginn)**
 - **arbeitsEnde = MIN(stempelEnde, sollEnde)**
 - **abrechenbar = arbeitsEnde - arbeitsBeginn - sollPause**
3. Nicht abrechenbare Zeit berechnen:
 - **vorSoll = sollBeginn - stempelBeginn** (wenn positiv)

- `nachSoll = stempelEnde - sollEnde` (wenn positiv)
- `nichtAbrechenbar = vorSoll + nachSoll`

Wichtig: Alle negativen Werte auf 0 setzen!

Hilfsfunktion: `berechneZuschlage(zeiterfassung, projektsoll, cuttingResult)`

Logik:

1. Prüfe Zuschlagsberechtigungs-Flags der Zeiterfassung
2. Wenn alle Flags `false` → return alle Zuschläge = 0
3. **Nacharbeit** (wenn `zuschlagNacht = true`):
 - Finde Überschneidung zwischen **abrechenbarer Zeit** und Nachtzeit
 - Nachtzeit-Fenster aus Zeitmodell laden (`nachtBeginn` bis `nachtEnde`)
 - Wenn `nachtEnde < nachtBeginn` → Nachtende ist am Folgetag
 - Berechne Überschneidung in Stunden
4. **Sonntagsarbeit** (wenn `zuschlagSonntag = true`):
 - Prüfe ob Datum ein Sonntag ist (`weekday === 7`)
 - Wenn ja: Gesamte abrechenbare Zeit ist Sonntagsarbeit
5. **Feiertagsarbeit** (wenn `zuschlagFeiertag = true`):
 - Prüfe ob Datum ein Feiertag ist (Datenbank-Abfrage in `Feiertag` Tabelle)
 - Berücksichtige Kanton des Mitarbeiters
 - Wenn ja: Gesamte abrechenbare Zeit ist Feiertagsarbeit
6. **Samstagsarbeit** (nur Deutschland, wenn `zuschlagSamstag = true`):
 - Prüfe ob Datum ein Samstag ist (`weekday === 6`)
 - Prüfe ob Land des Mitarbeiters Deutschland ist
 - Wenn ja: Gesamte abrechenbare Zeit ist Samstagsarbeit

Wichtig: Zuschläge werden NUR auf die **abrechenbare Zeit** berechnet, nicht auf die gesamte gestempelte Zeit!

Output: Objekt mit:

- `nacht`: Nachtarbeitsstunden
- `sonntag`: Sonntagsarbeitsstunden
- `feiertag`: Feiertagsarbeitsstunden
- `samstag`: Samstagsarbeitsstunden (nur DE)

3. API Endpoints

POST `/zeiterfassung/:id/cutting`

- Führt Cutting für eine bestimmte Zeiterfassung durch
- Gibt `CuttingResult` zurück
- Use Case: Manuelles Cutting nachträglich

POST `/zeiterfassung/:id/auto-cut`

- Beendet Stempelzeit (setzt `stempelEnde = now()`)
- Führt automatisch Cutting durch
- Gibt Zeiterfassung + CuttingResult zurück
- Use Case: Ausstempeln mit automatischem Cutting

PATCH /zeiterfassung/:id/zuschlag-flags

- Aktualisiert Zuschlagsberechtigungs-Flags
 - Body: { `zuschlagNacht: boolean, zuschlagSonntag: boolean, ... , grund?: string` }
 - Use Case: Mitarbeiter markiert Nacharbeit als "nicht zuschlagsberechtigt"
-

4. Frontend-Komponenten

Komponente: **ZeiterfassungCuttingAnzeige**

Zeigt an:

- 1. Gestempelte Zeit:**
 - Beginn - Ende
 - Gesamtdauer in Stunden
- 2. Projektsoll:**
 - Soll-Beginn - Soll-Ende
 - Soll-Arbeitszeit in Stunden
- 3. Abrechenbare Zeit:**
 - In grün/success-Farbe
 - Stunden
- 4. Nicht abrechenbare Zeit** (wenn > 0):
 - In gelb/warning-Farbe
 - Stunden
 - Hinweis: "X min zu früh" und/oder "Y min zu spät"
- 5. Zuschläge** (wenn vorhanden):
 - Liste der zuschlagsberechtigten Stunden
 - "Nacht: X.Xh (+25%)"
 - "Sonntag: X.Xh (+50%)"
 - "Feiertag: X.Xh (+100%)"

Komponente: **ZuschlagFlagEditor**

Ermöglicht:

- Checkboxes für `zuschlagNacht`, `zuschlagSonntag`, `zuschlagFeiertag`, `zuschlagSamstag`
- Textfeld für `zuschlagGrund` (wenn mindestens ein Flag auf `false`)
- Speichern-Button
- Hinweis: "Wenn du diese Stunden aus privaten Gründen gearbeitet hast, kannst du Zuschläge deaktivieren"

Komponente: **ProjektsollKonfigurator (Admin)**

Ermöglicht:

- Projektsoll für ein Projekt konfigurieren
 - Felder: `sollBeginn`, `sollEnde`, `sollPauseDauer`, `sollArbeitszeit`
 - Toggle: `cuttingAktiv`
 - Numerisches Feld: `cuttingTolerance` (Minuten)
-

5. Geschäftsregeln & Edge Cases**Edge Case: Mitarbeiter stempelt komplett außerhalb Projektsoll****Beispiel:**

- Projektsoll: 08:00 - 17:00
- Mitarbeiter stempelt: 18:00 - 20:00 (private Nacharbeit)

Verhalten:

- `abrechenbar` = 0
- `nichtAbrechenbar` = 2h
- `nachSoll` = 2h
- Warnung im UI: "Komplett außerhalb Projektsoll!"

Edge Case: Projekt hat kein Sollzeit definiert**Verhalten:**

- Fallback auf Zeitmodell des Mitarbeiters
- Standard 08:00 - (08:00 + tagesSollStunden)
- Hinweis im UI: "Projektsoll nicht definiert, verwende Zeitmodell"

Edge Case: Stempelzeit noch nicht beendet**Verhalten:**

- Cutting-Funktion wirft Fehler
- Frontend zeigt "Bitte zuerst ausstempeln"
- Oder: Live-Preview des Cuttings basierend auf aktuellem Zeitpunkt

Edge Case: Nachtarbeit über Mitternacht**Beispiel:**

- Nachtzeit: 22:00 - 06:00
- Arbeit: 23:00 - 02:00 (nächster Tag)

Verhalten:

- Berechne Überschneidung korrekt über Mitternacht hinweg
- 23:00 - 00:00 (1h) + 00:00 - 02:00 (2h) = 3h Nachtarbeit

Toleranz-Funktion

Wenn **cuttingTolerance** > 0:

- Beispiel: Toleranz = 15 Minuten
- Mitarbeiter kommt 10 Minuten zu früh → wird NICHT gecuted (innerhalb Toleranz)
- Mitarbeiter kommt 20 Minuten zu früh → 5 Minuten werden gecuted (20min - 15min Toleranz)

Implementierung:

- **vorSollEffektiv** = MAX(0, vorSoll - tolerance)
 - **nachSollEffektiv** = MAX(0, nachSoll - tolerance)
-

6. Testing-Szenarien

Implementiere Unit Tests für folgende Szenarien:

1. **Standard-Fall:** Mitarbeiter innerhalb Projektsoll
 2. **Zu früh:** 30min vor Projektsoll-Beginn
 3. **Zu spät:** 45min nach Projektsoll-Ende
 4. **Beides:** Zu früh UND zu spät
 5. **Komplett außerhalb:** Keine Überschneidung
 6. **Nachtarbeit:** Überschneidung mit Nachtzeit
 7. **Sonntagsarbeit:** Arbeit am Sonntag
 8. **Feiertagsarbeit:** Arbeit an Feiertag
 9. **Nicht zuschlagsberechtigt:** Flags auf false
 10. **Toleranz:** Mit verschiedenen Toleranzwerten
 11. **Kein Projektsoll:** Fallback auf Zeitmodell
 12. **Über Mitternacht:** Nachtarbeit über Mitternacht
-

7. Datenbank-Migration

Erstelle eine Prisma Migration die:

1. Zeitmodell-Felder hinzufügt
2. Projekt-Felder hinzufügt
3. Zeiterfassung-Felder hinzufügt
4. Indizes erstellt
5. Default-Werte setzt (z.B. **zuschlagNacht** = true)

Wichtig: Bestehende Daten dürfen nicht verloren gehen!

8. Dokumentation

Erstelle Markdown-Dokumentation die erklärt:

1. **Für Entwickler:** Wie das Cutting funktioniert (Algorithmus)
2. **Für Admins:** Wie Projektsoll konfiguriert wird

3. **Für Mitarbeiter:** Was "abrechenbar" vs "nicht abrechenbar" bedeutet
 4. **Für Mitarbeiter:** Wie Zuschlagsberechtigungs-Flags funktionieren
-

Prioritäten

Must-Have für MVP (Anfang März)

1. Prisma Schema-Erweiterung
2. Cutting-Service Basis-Implementierung
3. Projektsoll ermitteln (mit Fallback)
4. Abrechenbare Zeit berechnen
5. API Endpoint für Auto-Cutting
6. Einfache Frontend-Anzeige (Cutting-Ergebnis)

Nice-to-Have (später)

1. Zuschläge-Berechnung (Nacht, Sonntag, Feiertag)
 2. Zuschlagsberechtigungs-Flags
 3. Toleranz-Funktion
 4. Projektsoll-Konfigurator (Admin-UI)
 5. Live-Preview während Arbeit
 6. Detaillierte Reports
-

Performance-Überlegungen

- **Batch-Cutting:** Wenn viele Zeiterfassungen gecutet werden müssen (z.B. Monatsende), sollte das parallelisiert werden
 - **Caching:** Feiertags-Daten für aktuelles Jahr im Cache halten
 - **Lazy Loading:** Cutting nur durchführen wenn benötigt (nicht automatisch bei jedem Stempel)
 - **Indizes:** Zeiterfassungs-Queries nach Mitarbeiter+Datum optimieren
-

Sicherheit & Validierung

- **Nur Vorgesetzte** dürfen Zuschlagsberechtigungs-Flags ändern (außer Mitarbeiter selbst für "nicht berechtigt")
 - **Nur Admins** dürfen Projektsoll konfigurieren
 - **Audit-Log:** Alle Änderungen an Cutting-Ergebnissen protokollieren
 - **Validierung:** **abrechenbar** darf nie größer sein als **gesamtGestempelt**
-

Offene Fragen

1. Wie genau soll die Toleranz-Funktion arbeiten?
2. Gibt es Projekte wo Cutting NICHT aktiv sein soll?
3. Soll es automatisches Cutting beim Ausstempeln geben oder nur manuell?
4. Wer darf nachträglich Cutting-Ergebnisse ändern?

5. Was passiert bei vergessenen Stempelzeiten (Ticket-System)?

Nächste Schritte

1. **Schema erweitern:** Prisma Schema anpassen und Migration erstellen
 2. **Service implementieren:** CuttingService mit Basis-Funktionen
 3. **Testing:** Unit Tests für Standard-Szenarien
 4. **API:** Endpoints implementieren
 5. **Frontend:** Cutting-Anzeige Komponente
 6. **Review:** Code Review und Test mit Frank
 7. **Rollout:** Pilotgruppe Anfang März
-

Ziel: Anfang März 2026 einsatzbereit für erste Testgruppe